SQL MINI PROJECT

RETAIL STORE DATABASE

LEVEL 01 – BASICS

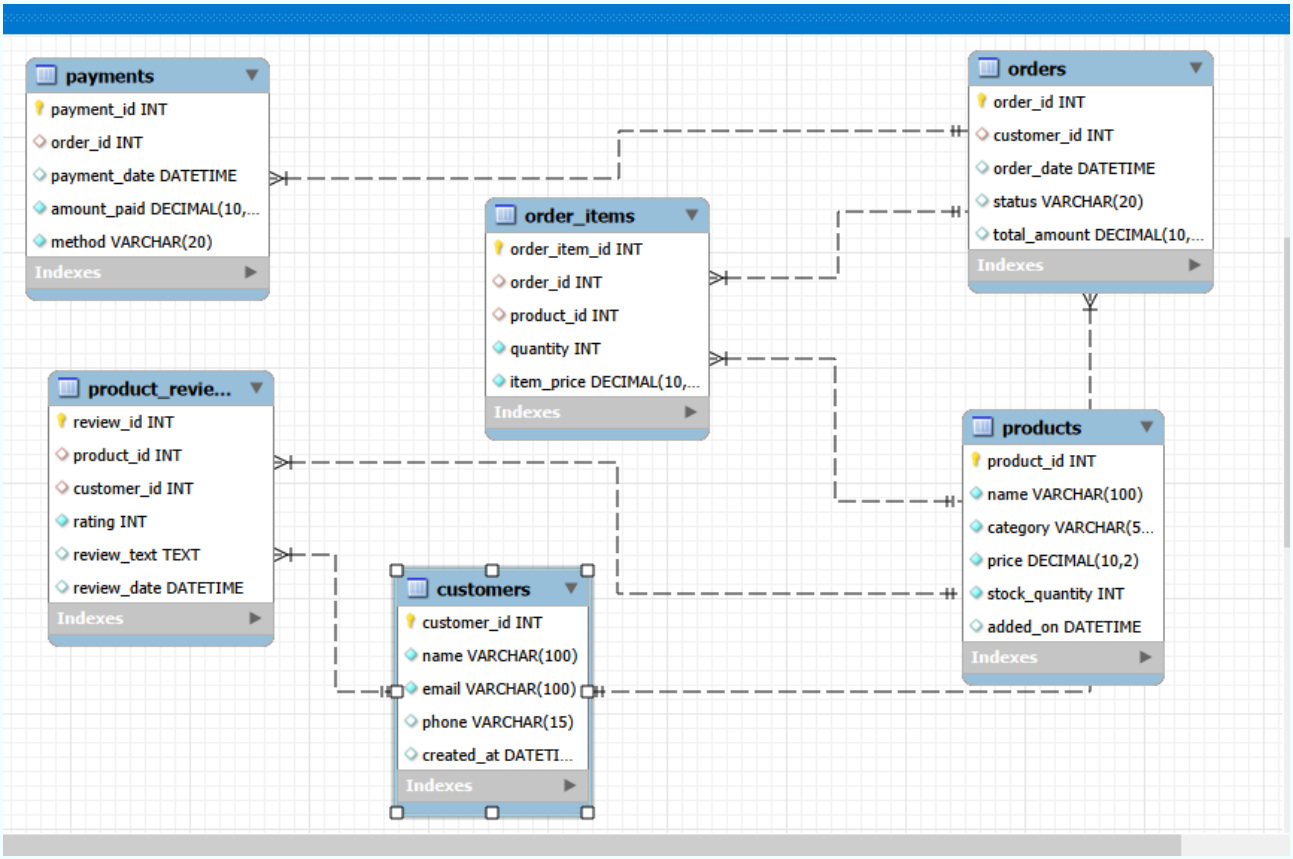
Introduction

The Retail Store Management System is designed to manage customers, products, orders, payments, and product reviews. The project uses SQL to store, retrieve, and analyze business data efficiently.

Objective

- Create a Retail Store Database.
- Store customer and product information.
- Manage orders and payments.
- Perform SQL queries for business analysis.

ER Diagram



✍ Onward: Table Structure

```sql
CREATE DATABASE retail_store_db;
USE retail_store_db;
```

```sql
CREATE TABLE customers(
        customer_id INT PRIMARY KEY AUTO_INCREMENT,
        name VARCHAR(100) NOT NULL,
        email VARCHAR(100) NOT NULL UNIQUE,
        phone VARCHAR(15),
        created_at DATETIME DEFAULT CURRENT_TIMESTAMP
        );
```

```sql
CREATE TABLE products(
        product_id INT PRIMARY KEY AUTO_INCREMENT,
        name VARCHAR(100) NOT NULL,
        category VARCHAR(50) NOT NULL,
        price DECIMAL(10,2) NOT NULL,
        stock_quantity INT NOT NULL DEFAULT 0,
        added_on DATETIME DEFAULT CURRENT_TIMESTAMP
        );
```

```sql
CREATE TABLE orders (
        order_id INT PRIMARY KEY AUTO_INCREMENT,
        customer_id INT,
        order_date DATETIME DEFAULT CURRENT_TIMESTAMP,
        status VARCHAR(20) DEFAULT 'Pending',
        total_amount DECIMAL(10,2),
        FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
        );
```

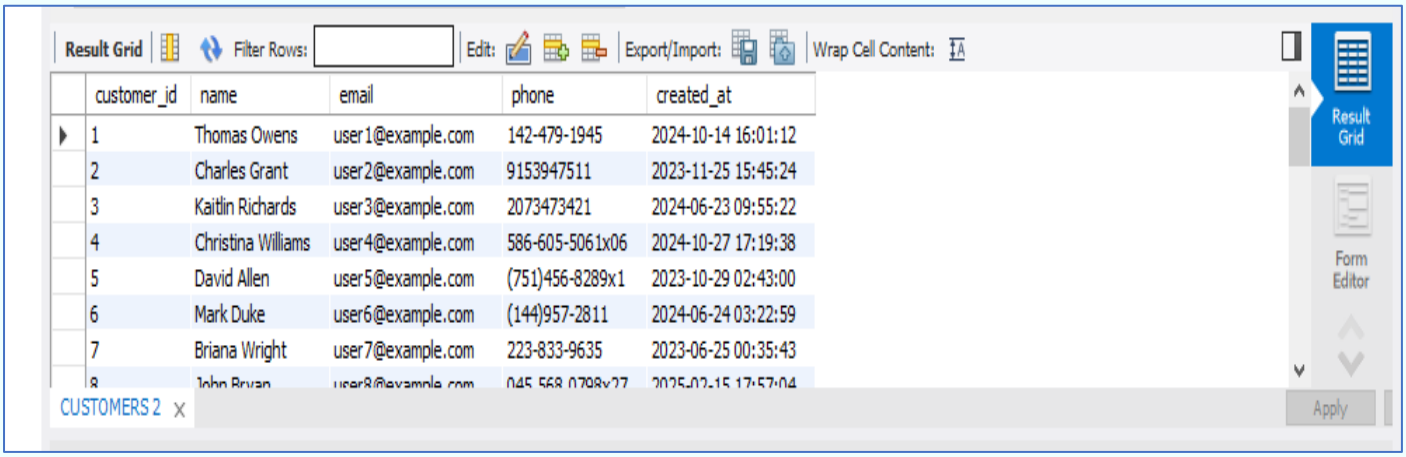
```sql
CREATE TABLE order_items (
        order_item_id INT PRIMARY KEY AUTO_INCREMENT,
        order_id INT,
        product_id INT,
        quantity INT NOT NULL CHECK (quantity > 0),
        item_price DECIMAL(10,2) NOT NULL,
        FOREIGN KEY (order_id) REFERENCES orders(order_id),
        FOREIGN KEY (product_id) REFERENCES products(product_id)
        );
```

```sql
CREATE TABLE payments (
        payment_id INT PRIMARY KEY AUTO_INCREMENT,
        order_id INT,
        payment_date DATETIME DEFAULT CURRENT_TIMESTAMP,
        amount_paid DECIMAL(10,2) NOT NULL CHECK (amount_paid > 0),
        method VARCHAR(20) NOT NULL,
        FOREIGN KEY (order_id) REFERENCES orders(order_id)
        );
```

```sql
CREATE TABLE product_reviews (
        review_id INT PRIMARY KEY AUTO_INCREMENT,
        product_id INT,
        customer_id INT,
        rating INT NOT NULL,
        review_text TEXT,
        review_date DATETIME DEFAULT CURRENT_TIMESTAMP,
        FOREIGN KEY (product_id) REFERENCES products(product_id),
        FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
        );
```

```sql
INSERT INTO customers (...);
INSERT INTO products (...);
INSERT INTO orders (...);
INSERT INTO payments (...);
INSERT INTO order_items (...);
INSERT INTO product_reviews (...);
```

*SELECT * FROM customers;*



customer_id	name	email	phone	created_at
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12
2	Charles Grant	user2@example.com	9153947511	2023-11-25 15:45:24
3	Kaitlin Richards	user3@example.com	2073473421	2024-06-23 09:55:22
4	Christina Williams	user4@example.com	586-605-5061x06	2024-10-27 17:19:38
5	David Allen	user5@example.com	(751)456-8289x1	2023-10-29 02:43:00
6	Mark Duke	user6@example.com	(144)957-2811	2024-06-24 03:22:59
7	Briana Wright	user7@example.com	223-833-9635	2023-06-25 00:35:43
8	John Bryan	user8@example.com	045 568 0798x27	2025-02-15 17:57:04

```
SELECT * FROM orders;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

order_id	customer_id	order_date	status	total_amount
1	20	2025-03-02 07:20:11	Delivered	9414.28
2	18	2024-10-09 18:08:21	Shipped	532.20
3	15	2025-05-08 00:08:27	Cancelled	5164.56
4	11	2024-09-19 22:16:13	Delivered	9469.78
5	12	2025-04-08 18:02:06	Pending	14501.86
6	29	2024-10-25 07:33:59	Cancelled	31050.17
7	22	2024-07-29 11:58:47	Shipped	3043.67
8	19	2024-07-30 22:49:49	Cancelled	32714.06

ORDERS 4

```
SELECT * FROM products;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

product_id	name	category	price	stock_quantity	added_on
1	Plant No	Home	639.43	152	2024-01-30 06:30:53
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
6	If Whatever	Electronics	177.40	64	2024-12-19 10:37:14
7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10

PRODUCTS 3

```
SELECT * FROM order_items;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

order_item_id	order_id	product_id	quantity	item_price
1	1	19	2	4707.14
2	2	6	3	177.40
3	3	9	3	985.19
4	3	23	1	2208.99
5	4	35	2	4734.89
6	5	19	1	4707.14
7	5	7	2	4897.36
8	6	7	3	4897.36

order_items 5

```
SELECT * FROM payments ;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

payment_id	order_id	payment_date	amount_paid	method
1	1	2025-03-02 08:16:11	9414.28	Credit Card
2	2	2024-10-09 19:00:21	532.20	Net Banking
3	3	2025-05-08 00:55:27	5164.56	Credit Card
4	4	2024-09-19 22:28:13	9469.78	UPI
5	5	2025-04-08 18:26:06	14501.86	UPI
6	6	2024-10-25 08:16:59	31050.17	UPI
7	7	2024-07-29 12:45:47	3043.67	UPI
8	8	2024-07-30 23:42:49	32714.06	Net Banking

payments 6

```
SELECT * FROM product_review ;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

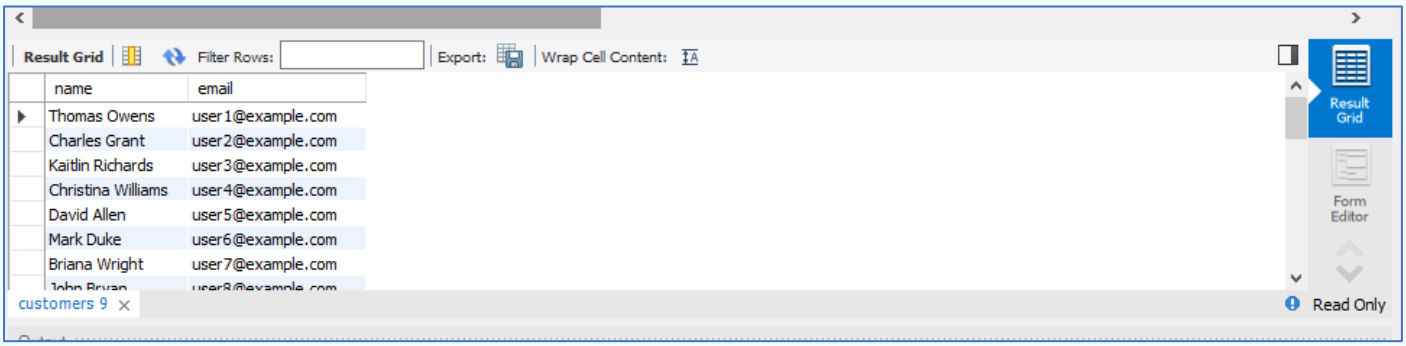
review_id	product_id	customer_id	rating	review_text	review_date
1	1	21	2	Great product, loved it!	2024-11-07 02:40:51
2	9	27	4	Product arrived damaged, had to return.	2025-06-08 02:40:51
3	42	21	5	Very fast delivery, happy with the purchase.	2024-10-01 02:40:51
4	13	28	2	Average quality, but acceptable.	2024-12-23 02:40:51
5	45	9	4	Good value for money.	2024-10-07 02:40:51
6	47	30	5	Product arrived damaged, had to return.	2024-07-06 02:40:51
7	6	25	4	Product arrived damaged, had to return.	2024-11-10 02:40:51
8	27	29	5	Great product, loved it!	2024-08-02 02:40:51

product_reviews 8

LEVEL 1: BASICS

1. Retrieve customer names and emails for email marketing

```
SELECT name, email
FROM customers;
```

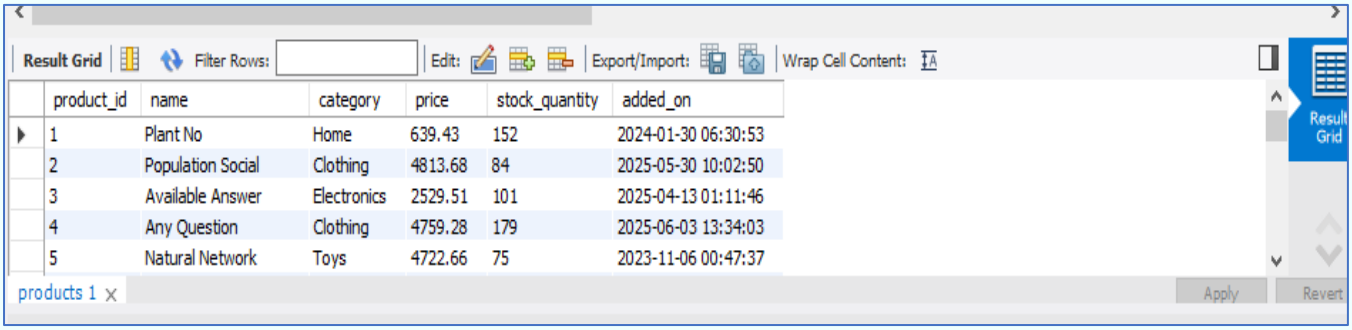


name	email
Thomas Owens	user1@example.com
Charles Grant	user2@example.com
Kaitlin Richards	user3@example.com
Christina Williams	user4@example.com
David Allen	user5@example.com
Mark Duke	user6@example.com
Briana Wright	user7@example.com

customers 9 | Read Only

2. View complete product catalog with all available details

```
SELECT * FROM products;
```

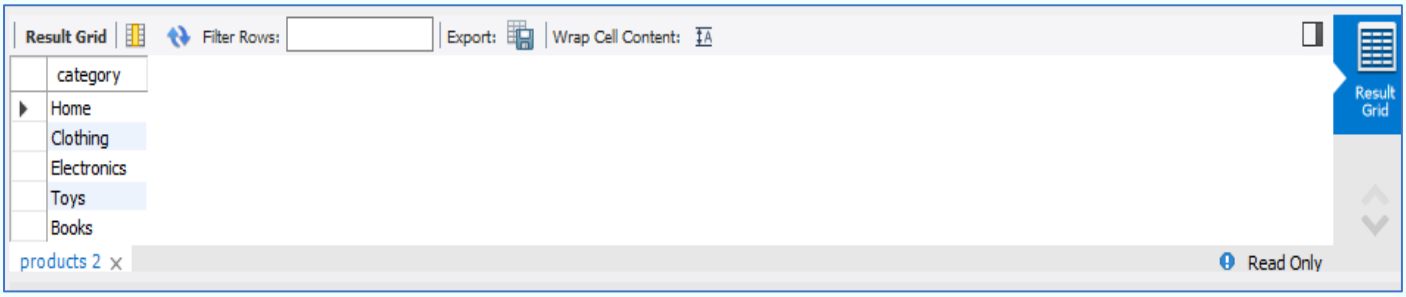


product_id	name	category	price	stock_quantity	added_on
1	Plant No	Home	639.43	152	2024-01-30 06:30:53
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37

products 1 | Apply | Revert

3. List all unique product categories

```
SELECT DISTINCT category from products;
```



category
Home
Clothing
Electronics
Toys
Books

products 2 | Read Only

4. Show all products priced above ₹1,000

```
SELECT * FROM products where price > 1000;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category
Home
Clothing
Electronics
Toys
Books

products 3 | Read Only

5. Display products within a mid-range price bracket (₹2,000 to ₹5,000)

```
SELECT * FROM products where price between 2000 and 5000;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

product_id	name	category	price	stock_quantity	added_on
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08

products 5 | Apply | Revert

6. Fetch data for specific customer IDs (e.g., from loyalty program list)

```
SELECT * FROM customers where customer_id in (2,8,15);
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

customer_id	name	email	phone	created_at
2	Charles Grant	user2@example.com	9153947511	2023-11-25 15:45:24
8	John Bryan	user8@example.com	045.568.0798x27	2025-02-15 17:57:04
15	Austin Flores	user15@example.com	329.901.1576x66	2024-06-13 09:03:42
NULL	NULL	NULL	NULL	NULL

7. Identify customers whose names start with the letter ‘A’

```
SELECT * FROM customers where name like 'A%' ;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

customer_id	name	email	phone	created_at
15	Austin Flores	user15@example.com	329.901.1576x66	2024-06-13 09:03:42
16	Amy Landry	user16@example.com	+1-278-019-3748	2024-02-28 17:51:50
19	Amanda Bright	user19@example.com	380.981.9798x69	2024-12-20 22:58:15
27	Adrienne Green	user27@example.com	530.644.8455x93	2023-08-22 01:55:29
NULL	NULL	NULL	NULL	NULL

customers 8 | Apply | Revert

8. List electronics products priced under ₹3,000

```
SELECT *
FROM products
WHERE category = 'Electronics'
 AND price < 3000;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

product_id	name	category	price	stock_quantity	added_on
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
6	If Whatever	Electronics	177.40	64	2024-12-19 10:37:14
15	Place Low	Electronics	723.97	46	2023-07-05 14:36:07
31	Series Page	Electronics	2070.37	83	2024-04-01 00:24:06
34	Despite Win	Electronics	1340.34	64	2024-11-27 06:55:45

products 9 | Apply | Revert

9. Display product names and prices in descending order of price

```
SELECT name, price FROM products
order by price DESC ;
```

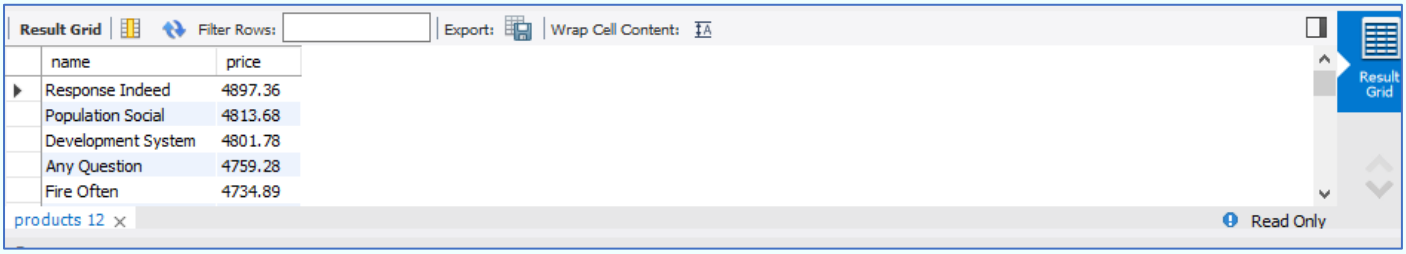
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	price
Response Indeed	4897.36
Population Social	4813.68
Development System	4801.78
Any Question	4759.28
Fire Often	4734.89

products 11 | Read Only

10. Display product names and prices, sorted by price and then by name

```
SELECT name, price FROM products
order by price DESC , name ASC ;
```

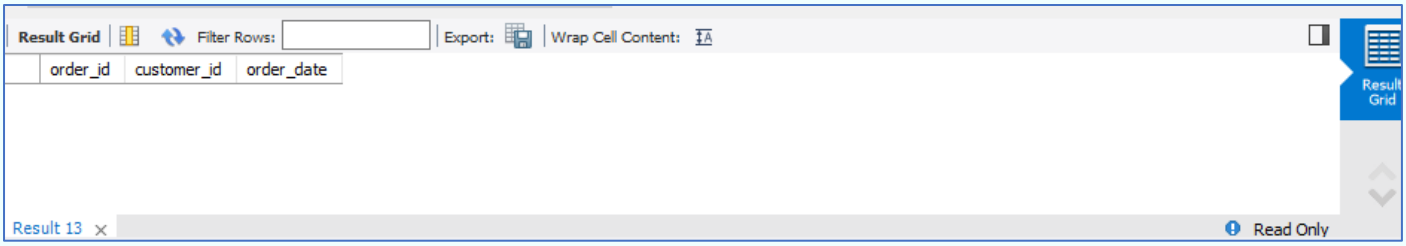


LEVEL 2:

FILTERING AND FORMATTING

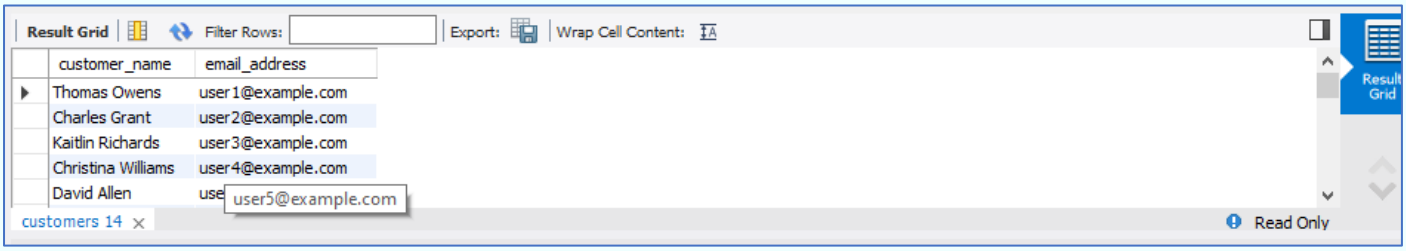
1. Retrieve orders where customer information is missing (possibly due to data migration or deletion)

```
SELECT o.order_id, o.customer_id, o.order_date
FROM orders o
LEFT JOIN customers c
ON o.customer_id = c.customer_id
WHERE c.customer_id IS NULL;
```



2. Display customer names and emails using column aliases for frontend readability

```
select name as customer_name, email as email_address
from customers;
```



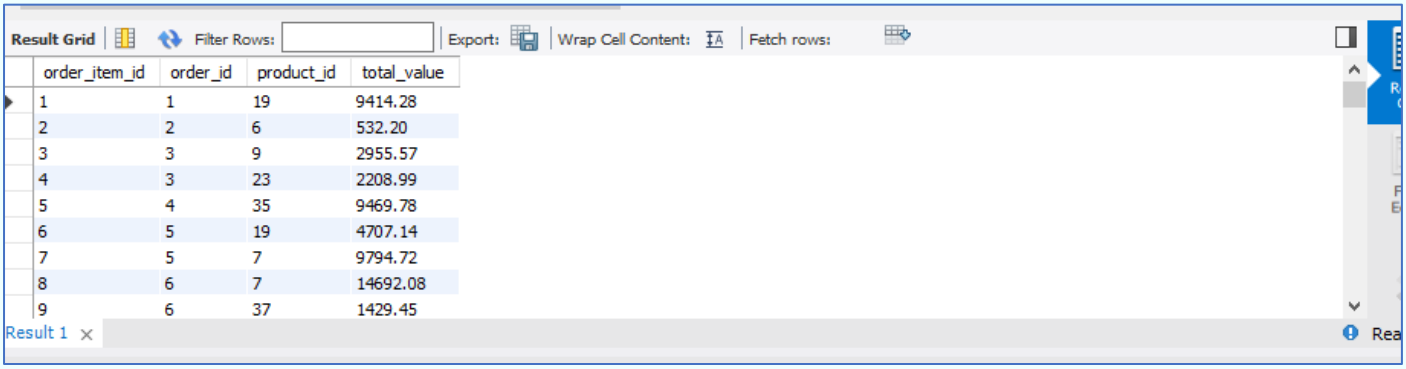
customer_name	email_address
Thomas Owens	user1@example.com
Charles Grant	user2@example.com
Kaitlin Richards	user3@example.com
Christina Williams	user4@example.com
David Allen	use

user5@example.com

customers 14 × Read Only

3. Calculate total value per item ordered by multiplying quantity and item price

```
SELECT order_item_id, order_id, product_id, quantity * item_price as total_value
from order_items;
```

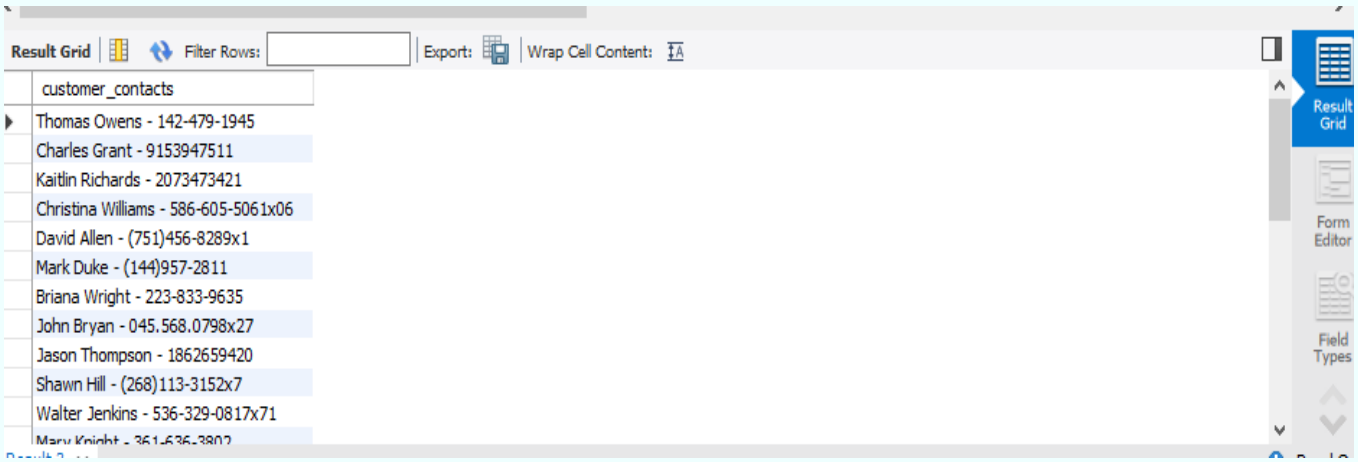


order_item_id	order_id	product_id	total_value
1	1	19	9414.28
2	2	6	532.20
3	3	9	2955.57
4	3	23	2208.99
5	4	35	9469.78
6	5	19	4707.14
7	5	7	9794.72
8	6	7	14692.08
9	6	37	1429.45

Result 1 ×

4. Combine customer name and phone number in a single column

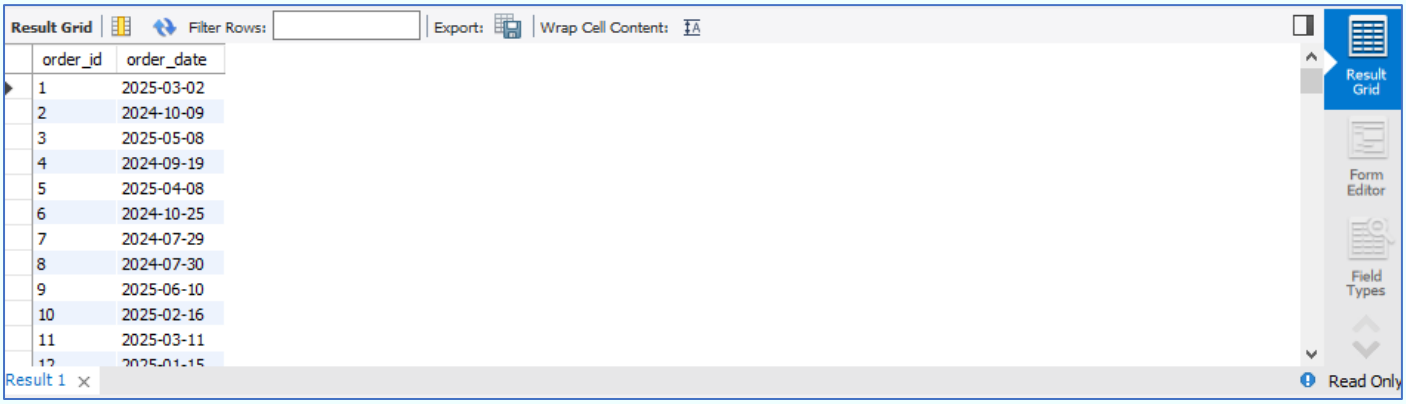
```
SELECT concat(
        name, ' - ',
  COALESCE (phone, 'Not Available')
  ) AS customer_contacts
  FROM customers;
```



customer_contacts
Thomas Owens - 142-479-1945
Charles Grant - 9153947511
Kaitlin Richards - 2073473421
Christina Williams - 586-605-5061x06
David Allen - (751)456-8289x1
Mark Duke - (144)957-2811
Briana Wright - 223-833-9635
John Bryan - 045.568.0798x27
Jason Thompson - 1862659420
Shawn Hill - (268)113-3152x7
Walter Jenkins - 536-329-0817x71
Mary Knight - 361-636-3802

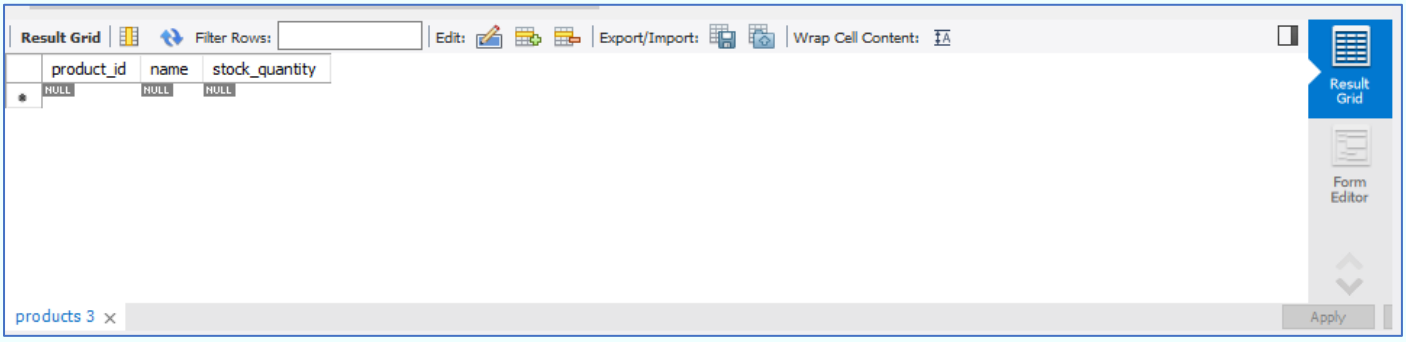
5. Extract only the date part from order timestamps for date-wise reporting

```
SELECT order_id,
       date(order_date) as order_date
  from orders;
```



6. List products that do not have any stock left

```
SELECT product_id, name, stock_quantity
FROM products
WHERE stock_quantity = 0;
```

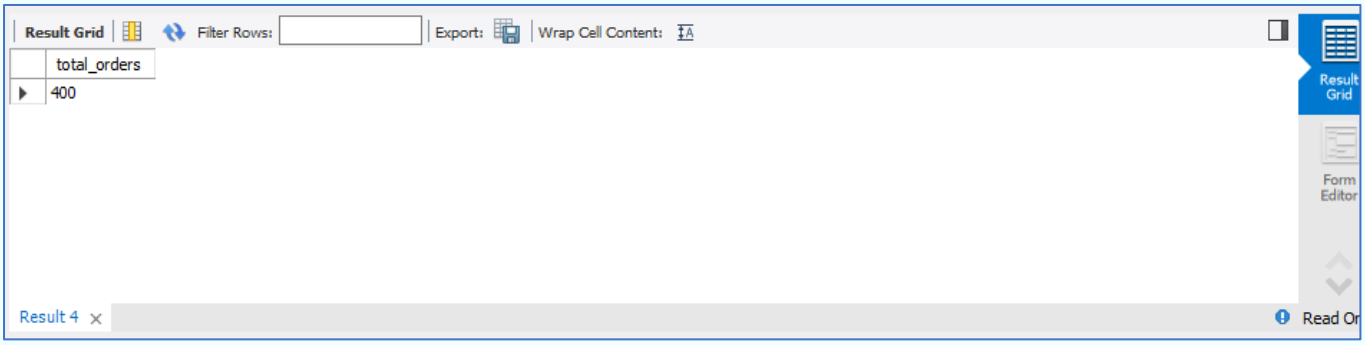


LEVEL 3:

AGGREGATIONS

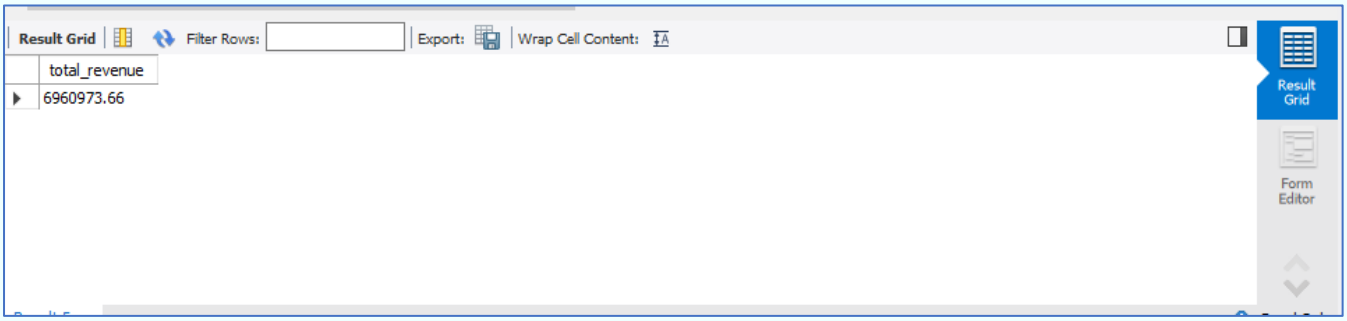
1. Count the total number of orders placed

```
SELECT COUNT(*) As total_orders
FROM orders ;
```



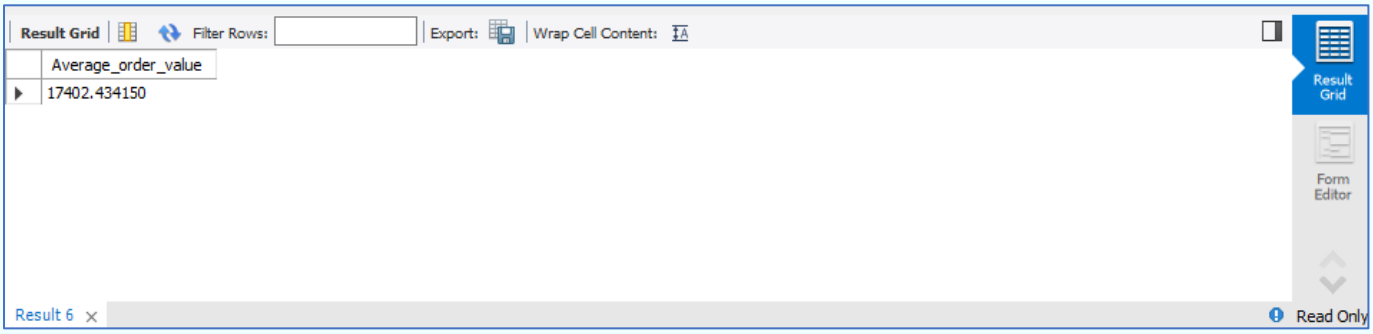
2. Calculate the total revenue collected from all orders

```
SELECT
   ROUND(SUM(amount_paid), 2) AS
total_payments_received
FROM payments;
```



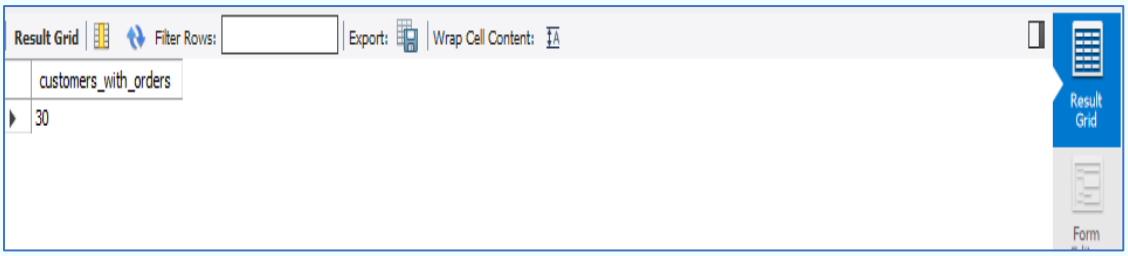
3. Calculate the average order value

```
SELECT AVG ( total_amount ) AS Average_order_value
FROM orders;
```



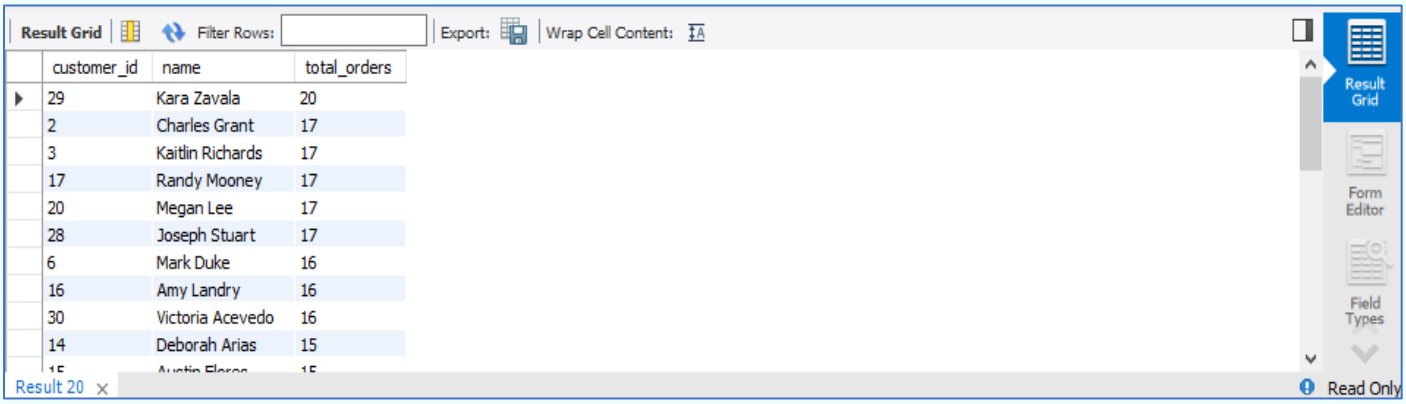
4. Count the number of customers who have placed at least one order

```
SELECT COUNT(DISTINCT customer_id) AS
customers_with_orders
FROM orders;
```



5. Find the number of orders placed by each customer

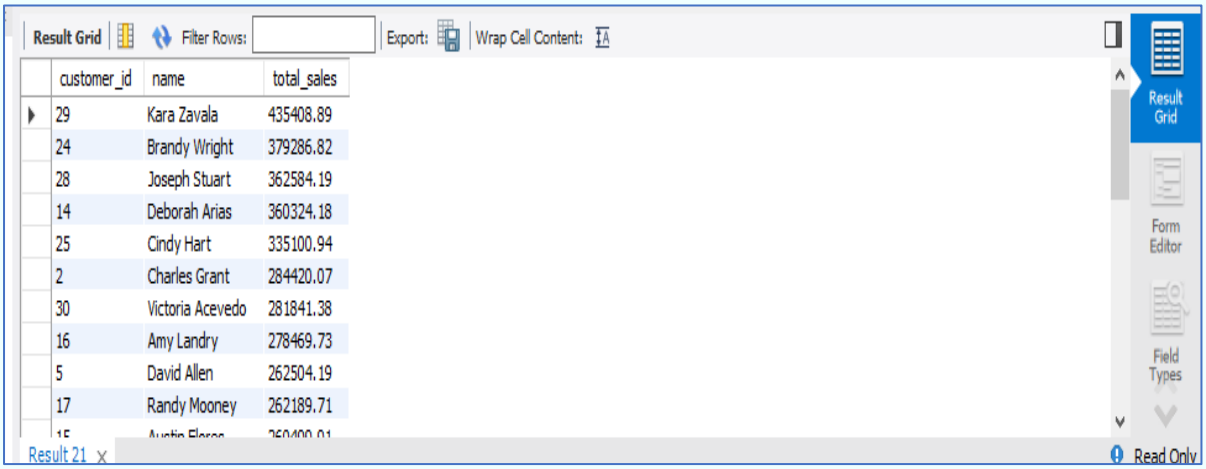
```
SELECT
   c.customer_id,
   c.name,
   COUNT(o.order_id) AS total_orders
FROM customers c
LEFT JOIN orders o
   ON c.customer_id = o.customer_id
GROUP BY
   c.customer_id,
   c.name
ORDER BY total_orders DESC;
```



customer_id	name	total_orders
29	Kara Zavala	20
2	Charles Grant	17
3	Kaitlin Richards	17
17	Randy Mooney	17
20	Megan Lee	17
28	Joseph Stuart	17
6	Mark Duke	16
16	Amy Landry	16
30	Victoria Acevedo	16
14	Deborah Arias	15

6. Find total sales amount made by each customer

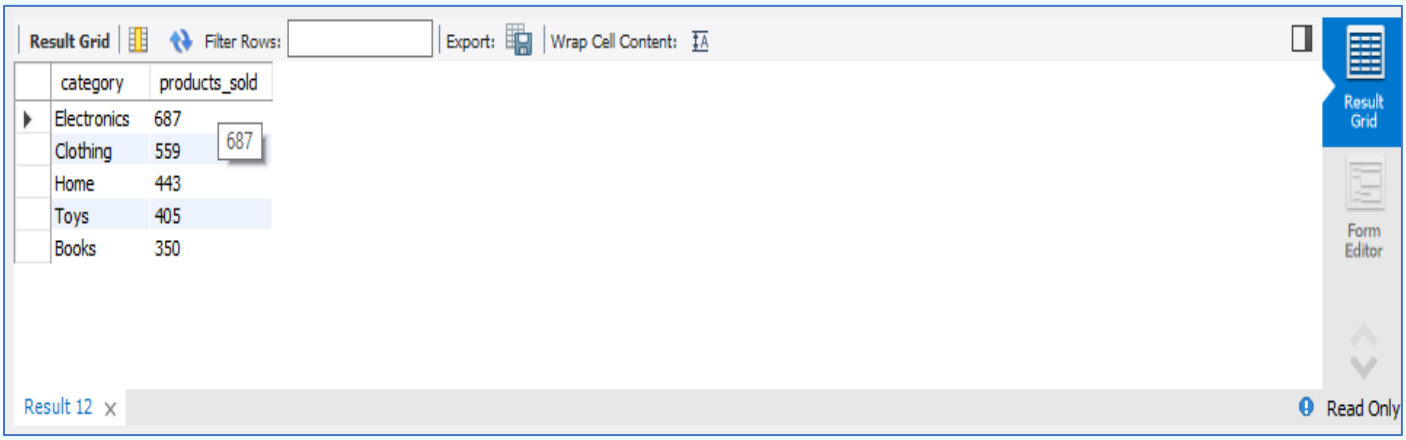
```
SELECT
    c.customer_id,
    c.name,
    SUM(o.total_amount) AS total_sales
FROM customers c
JOIN orders o
    ON c.customer_id = o.customer_id
GROUP BY
    c.customer_id,
    c.name
ORDER BY total_sales DESC;
```



customer_id	name	total_sales
29	Kara Zavala	435408.89
24	Brandy Wright	379286.82
28	Joseph Stuart	362584.19
14	Deborah Arias	360324.18
25	Cindy Hart	335100.94
2	Charles Grant	284420.07
30	Victoria Acevedo	281841.38
16	Amy Landry	278469.73
5	David Allen	262504.19
17	Randy Mooney	262189.71

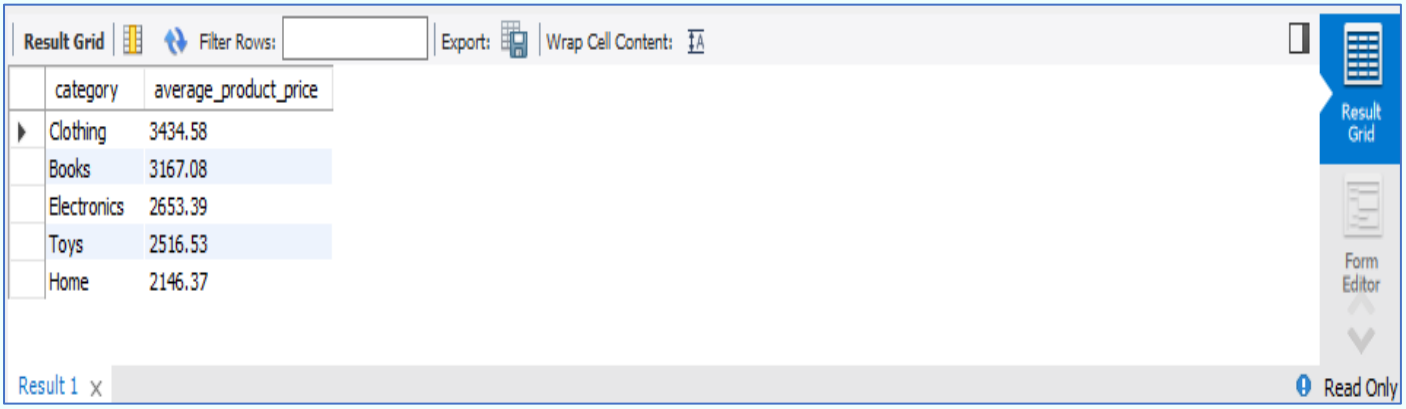
7. List the number of products sold per category

```
SELECT p.category, SUM(oi.quantity) AS products_sold
FROM products p
JOIN order_items oi
ON p.product_id = oi.product_id
GROUP BY p.category
ORDER BY products_sold DESC;
```



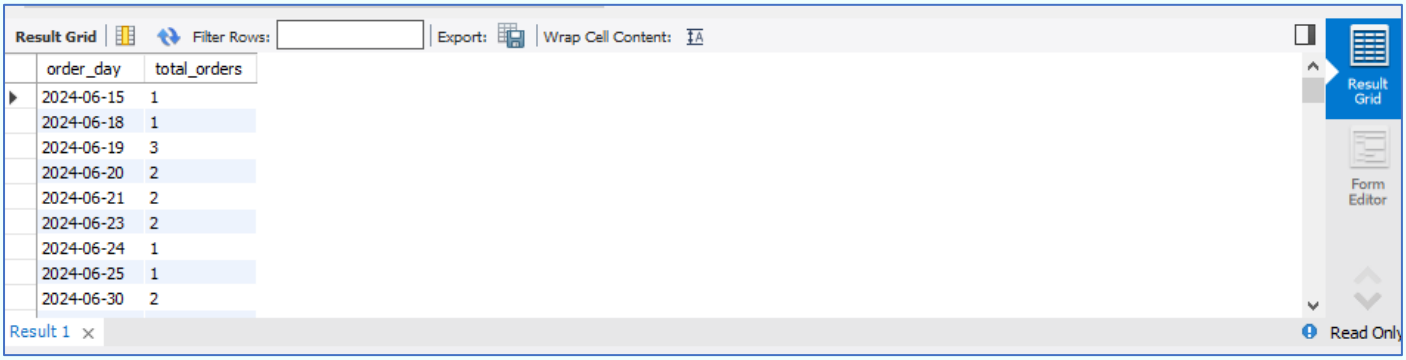
8. Find the average item price per category

```
SELECT category,
       ROUND(AVG(price), 2) AS average_product_price
  FROM products
  GROUP BY category
  ORDER BY average_product_price DESC;
```



9. Show number of orders placed per day

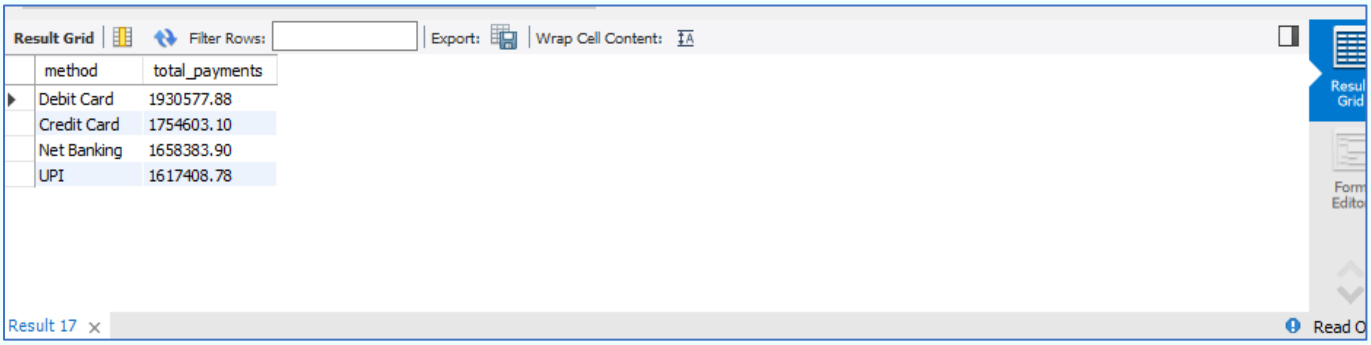
```
SELECT DATE(order_date) AS order_day,
       COUNT(order_id) AS total_orders
FROM orders
GROUP BY DATE(order_date)
ORDER BY order_day;
```



order_day	total_orders
2024-06-15	1
2024-06-18	1
2024-06-19	3
2024-06-20	2
2024-06-21	2
2024-06-23	2
2024-06-24	1
2024-06-25	1
2024-06-30	2

10. List total payments received per payment method

```
SELECT method,
    ROUND(SUM(amount_paid), 2) AS total_payments
FROM payments
GROUP BY method
ORDER BY total_payments DESC;
```

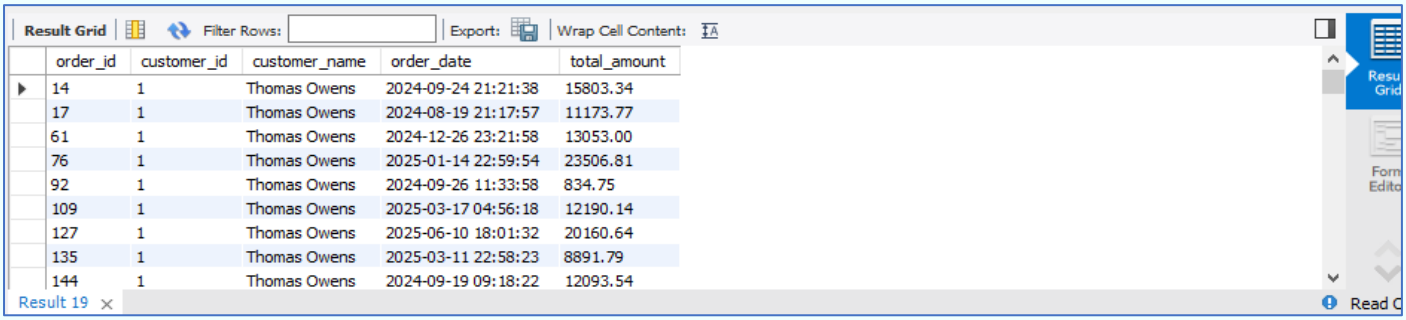


method	total_payments
Debit Card	1930577.88
Credit Card	1754603.10
Net Banking	1658383.90
UPI	1617408.78

LEVEL 4: MULTI-TABLE QUERIES (JOINS)

1. Retrieve order details along with the customer name (INNER JOIN)

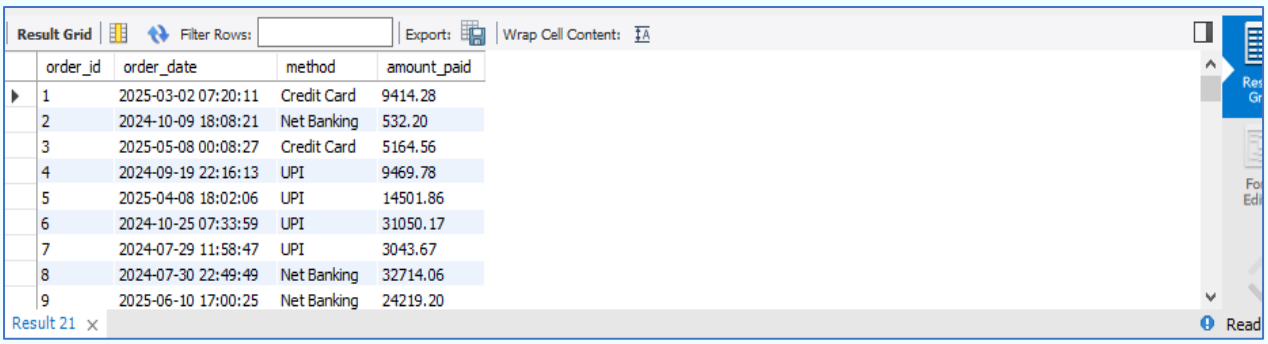
```
SELECT o.order_id, c.customer_id, c.name AS customer_name, o.order_date, o.total_amount
FROM orders o
INNER JOIN customers c
ON o.customer_id = c.customer_id;
```



order_id	customer_id	customer_name	order_date	total_amount
14	1	Thomas Owens	2024-09-24 21:21:38	15803.34
17	1	Thomas Owens	2024-08-19 21:17:57	11173.77
61	1	Thomas Owens	2024-12-26 23:21:58	13053.00
76	1	Thomas Owens	2025-01-14 22:59:54	23506.81
92	1	Thomas Owens	2024-09-26 11:33:58	834.75
109	1	Thomas Owens	2025-03-17 04:56:18	12190.14
127	1	Thomas Owens	2025-06-10 18:01:32	20160.64
135	1	Thomas Owens	2025-03-11 22:58:23	8891.79
144	1	Thomas Owens	2024-09-19 09:18:22	12093.54

2. Get list of products that have been sold (INNER JOIN with order_items)

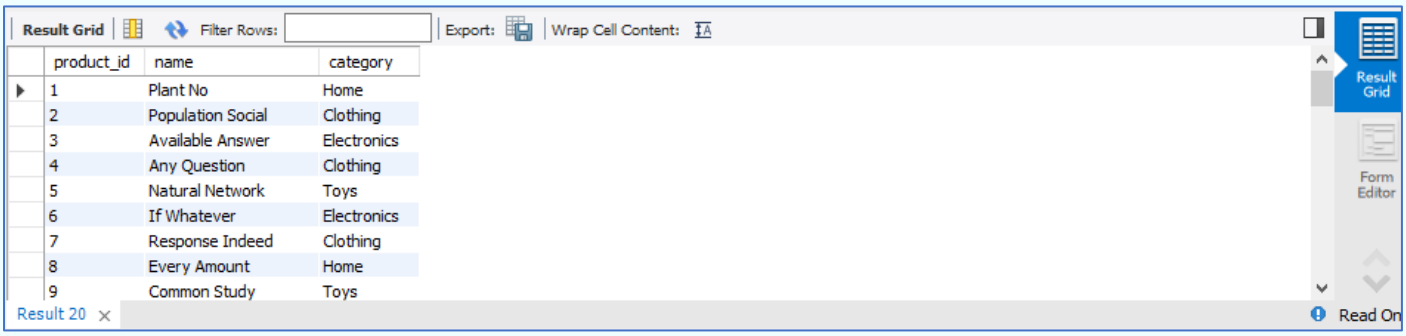
```
SELECT
   o.order_id,
   o.order_date,
   p.method,
   p.amount_paid
FROM orders o
INNER JOIN payments p
   ON o.order_id = p.order_id;
```



order_id	order_date	method	amount_paid
1	2025-03-02 07:20:11	Credit Card	9414.28
2	2024-10-09 18:08:21	Net Banking	532.20
3	2025-05-08 00:08:27	Credit Card	5164.56
4	2024-09-19 22:16:13	UPI	9469.78
5	2025-04-08 18:02:06	UPI	14501.86
6	2024-10-25 07:33:59	UPI	31050.17
7	2024-07-29 11:58:47	UPI	3043.67
8	2024-07-30 22:49:49	Net Banking	32714.06
9	2025-06-10 17:00:25	Net Banking	24219.20

3. List all orders with their payment method (INNER JOIN)

```
SELECT DISTINCT p.product_id, p.name, p.category
FROM products p
INNER JOIN order_items oi
ON p.product_id = oi.product_id;
```



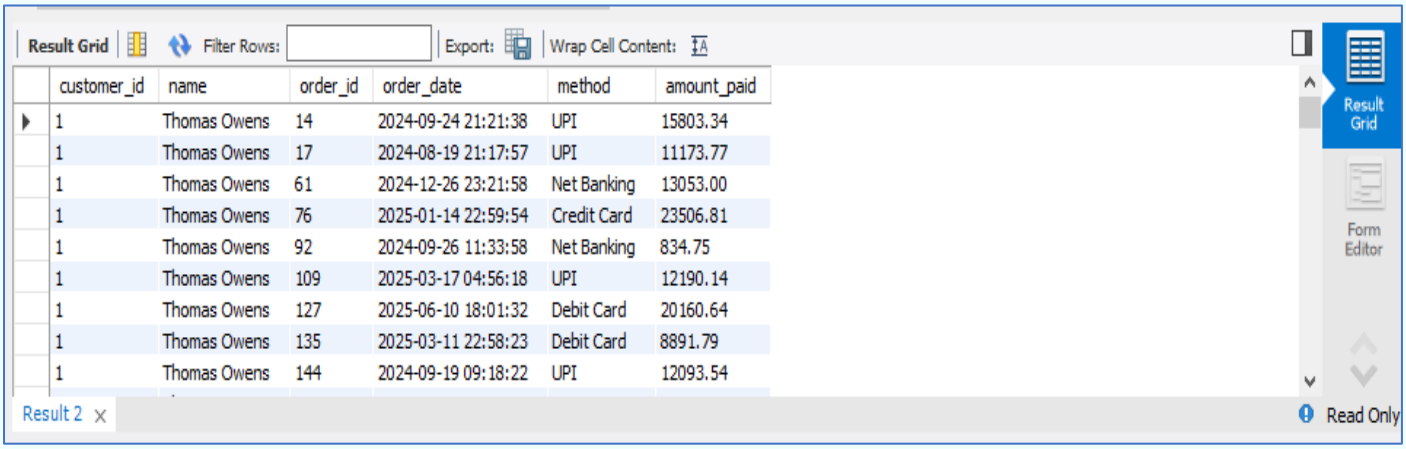
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_id	name	category
1	Plant No	Home
2	Population Social	Clothing
3	Available Answer	Electronics
4	Any Question	Clothing
5	Natural Network	Toys
6	If Whatever	Electronics
7	Response Indeed	Clothing
8	Every Amount	Home
9	Common Study	Toys

Result 20 | Result Grid | Form Editor | Read On

4. Get list of customers and their orders (LEFT JOIN)

```
SELECT
    c.customer_id,
    c.name,
    o.order_id,
    o.order_date,
    p.method,
    p.amount_paid
FROM customers c
INNER JOIN orders o
    ON c.customer_id = o.customer_id
INNER JOIN payments p
    ON o.order_id = p.order_id;
```



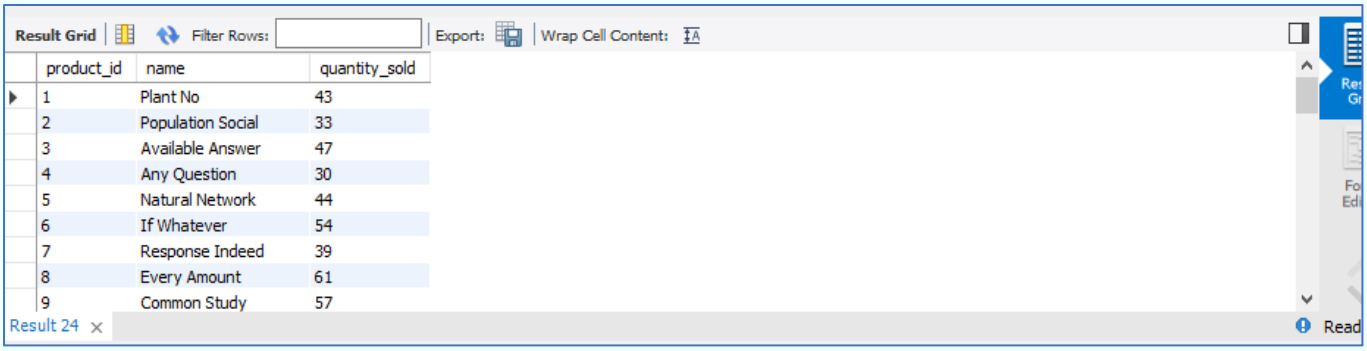
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	name	order_id	order_date	method	amount_paid
1	Thomas Owens	14	2024-09-24 21:21:38	UPI	15803.34
1	Thomas Owens	17	2024-08-19 21:17:57	UPI	11173.77
1	Thomas Owens	61	2024-12-26 23:21:58	Net Banking	13053.00
1	Thomas Owens	76	2025-01-14 22:59:54	Credit Card	23506.81
1	Thomas Owens	92	2024-09-26 11:33:58	Net Banking	834.75
1	Thomas Owens	109	2025-03-17 04:56:18	UPI	12190.14
1	Thomas Owens	127	2025-06-10 18:01:32	Debit Card	20160.64
1	Thomas Owens	135	2025-03-11 22:58:23	Debit Card	8891.79
1	Thomas Owens	144	2024-09-19 09:18:22	UPI	12093.54

Result 2 | Result Grid | Form Editor | Read Only

5. List all products along with order item quantity (LEFT JOIN)

```
SELECT p.product_id, p.name,
   IFNULL(SUM(oi.quantity),0) AS quantity_sold
FROM products p
LEFT JOIN order_items oi
ON p.product_id = oi.product_id
GROUP BY p.product_id, p.name;
```



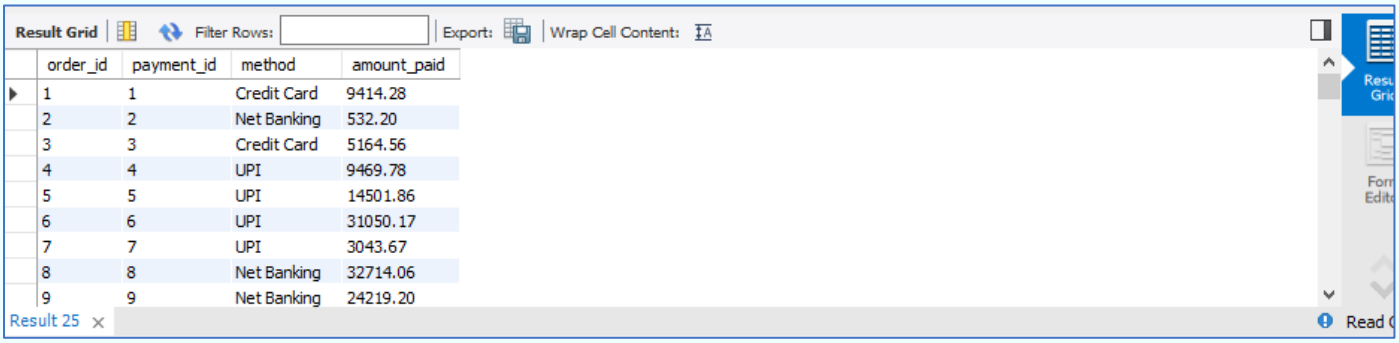
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_id	name	quantity_sold
1	Plant No	43
2	Population Social	33
3	Available Answer	47
4	Any Question	30
5	Natural Network	44
6	If Whatever	54
7	Response Indeed	39
8	Every Amount	61
9	Common Study	57

Result 24 | Read

6. List all payments including those with no matching orders (RIGHT JOIN)

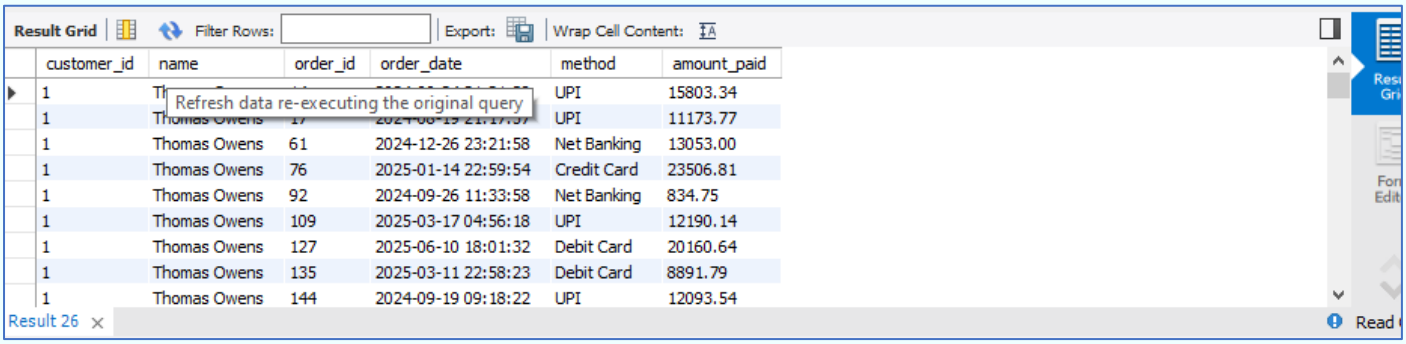
```
SELECT o.order_id, p.payment_id, p.method, p.amount_paid
FROM orders o
RIGHT JOIN payments p
ON o.order_id = p.order_id;
```



order_id	payment_id	method	amount_paid
1	1	Credit Card	9414.28
2	2	Net Banking	532.20
3	3	Credit Card	5164.56
4	4	UPI	9469.78
5	5	UPI	14501.86
6	6	UPI	31050.17
7	7	UPI	3043.67
8	8	Net Banking	32714.06
9	9	Net Banking	24219.20

7. Combine data from three tables: customer, order, and payment

```
SELECT c.customer_id, c.name, o.order_id, o.order_date, p.method,
p.amount_paid
FROM customers c
INNER JOIN orders o
ON c.customer_id = o.customer_id
INNER JOIN payments p
ON o.order_id = p.order_id;
```



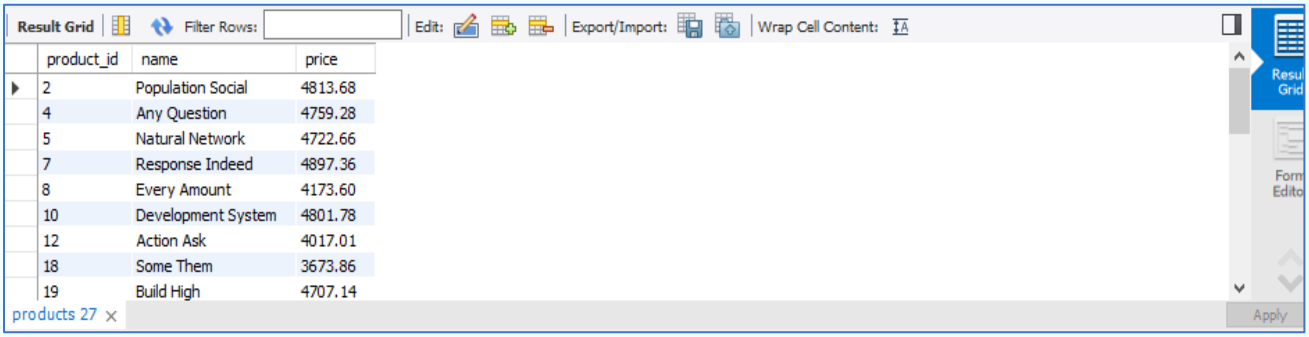
customer_id	name	order_id	order_date	method	amount_paid
1	[illegible]	[illegible]	[illegible]	UPI	15803.34
1	[illegible]	[illegible]	[illegible]	UPI	11173.77
1	Thomas Owens	61	2024-12-26 23:21:58	Net Banking	13053.00
1	Thomas Owens	76	2025-01-14 22:59:54	Credit Card	23506.81
1	Thomas Owens	92	2024-09-26 11:33:58	Net Banking	834.75
1	Thomas Owens	109	2025-03-17 04:56:18	UPI	12190.14
1	Thomas Owens	127	2025-06-10 18:01:32	Debit Card	20160.64
1	Thomas Owens	135	2025-03-11 22:58:23	Debit Card	8891.79
1	Thomas Owens	144	2024-09-19 09:18:22	UPI	12093.54

Refresh data re-executing the original query

LEVEL 5: SUBQUERIES (INNER QUERIES)

1. List all products priced above the average product price

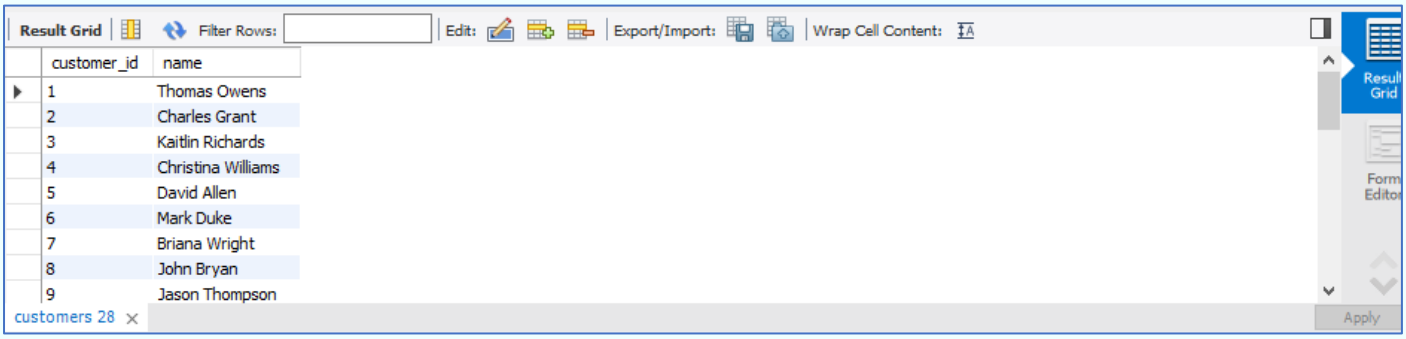
```
SELECT product_id, name, price
FROM products
WHERE price >
        (SELECT AVG(price)
        FROM products
);
```



product_id	name	price
2	Population Social	4813.68
4	Any Question	4759.28
5	Natural Network	4722.66
7	Response Indeed	4897.36
8	Every Amount	4173.60
10	Development System	4801.78
12	Action Ask	4017.01
18	Some Them	3673.86
19	Build High	4707.14

2. Find customers who have placed at least one order

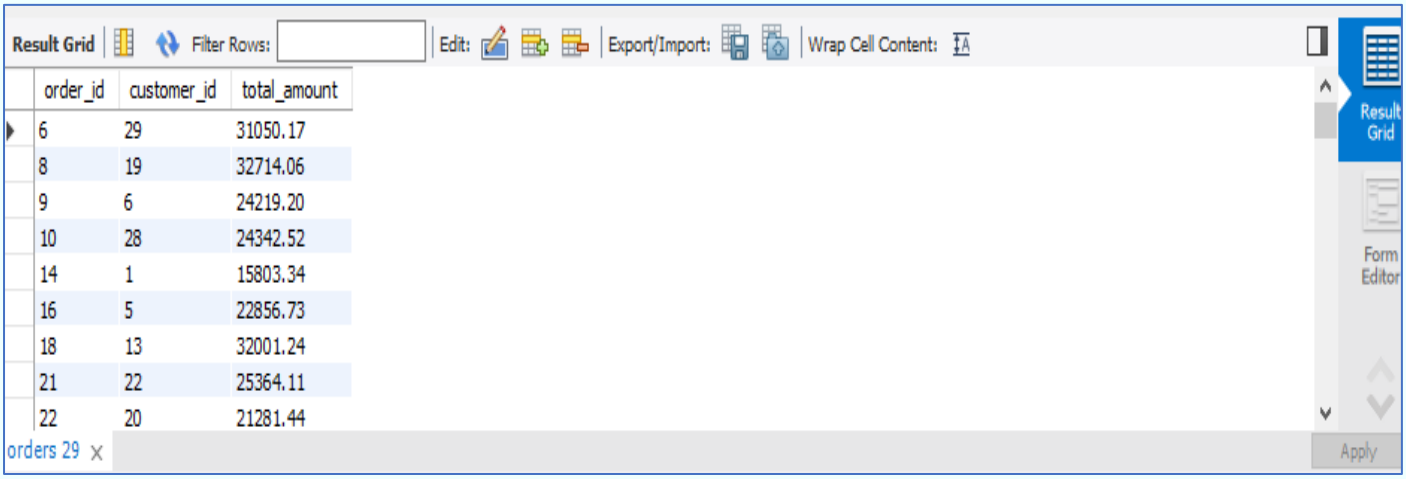
```
SELECT customer_id, name
FROM customers
WHERE customer_id IN
                (SELECT customer_id
                FROM orders
);
```



customer_id	name
1	Thomas Owens
2	Charles Grant
3	Kaitlin Richards
4	Christina Williams
5	David Allen
6	Mark Duke
7	Briana Wright
8	John Bryan
9	Jason Thompson

3. Show orders whose total amount is above the average for that customer

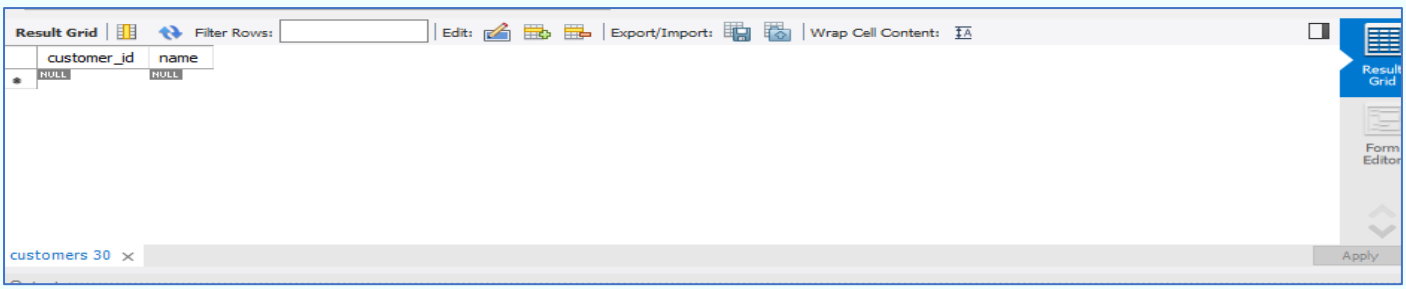
```
SELECT o.order_id, o.customer_id, o.total_amount
FROM orders o
WHERE o.total_amount >
                (SELECT AVG(total_amount)
                FROM orders
                WHERE customer_id = o.customer_id
);
```



order_id	customer_id	total_amount
6	29	31050.17
8	19	32714.06
9	6	24219.20
10	28	24342.52
14	1	15803.34
16	5	22856.73
18	13	32001.24
21	22	25364.11
22	20	21281.44

4. Display customers who haven't placed any orders

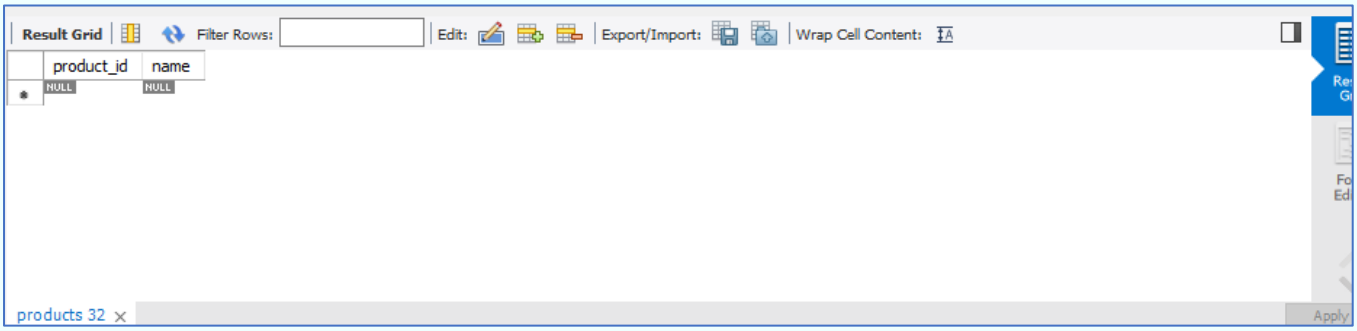
```
SELECT
   c.customer_id,
   c.name
FROM customers c
WHERE NOT EXISTS (
   SELECT 1
   FROM orders o
   WHERE o.customer_id = c.customer_id
);
```



customer_id	name
NULL	NULL

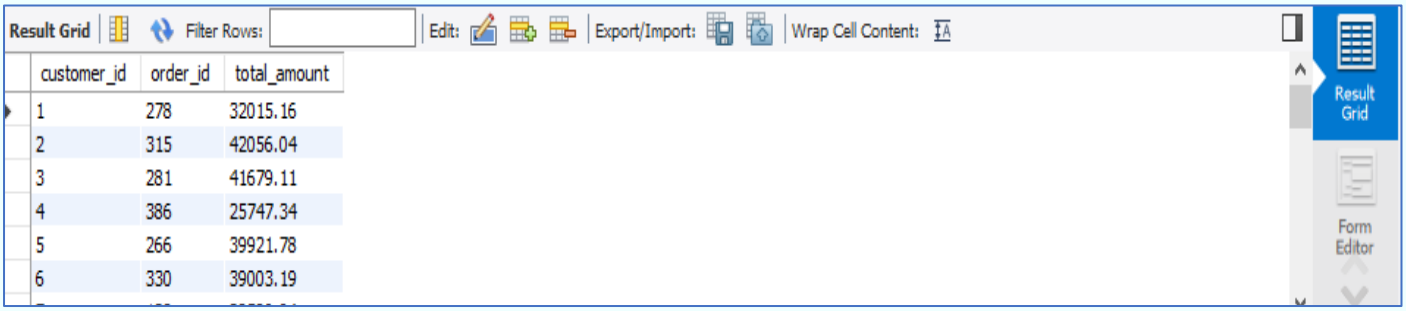
5. Show products that were never ordered

```
SELECT
   p.product_id,
   p.name
FROM products p
WHERE NOT EXISTS (
   SELECT 1
   FROM order_items oi
   WHERE oi.product_id = p.product_id
);
```



6. Show highest value order per customer

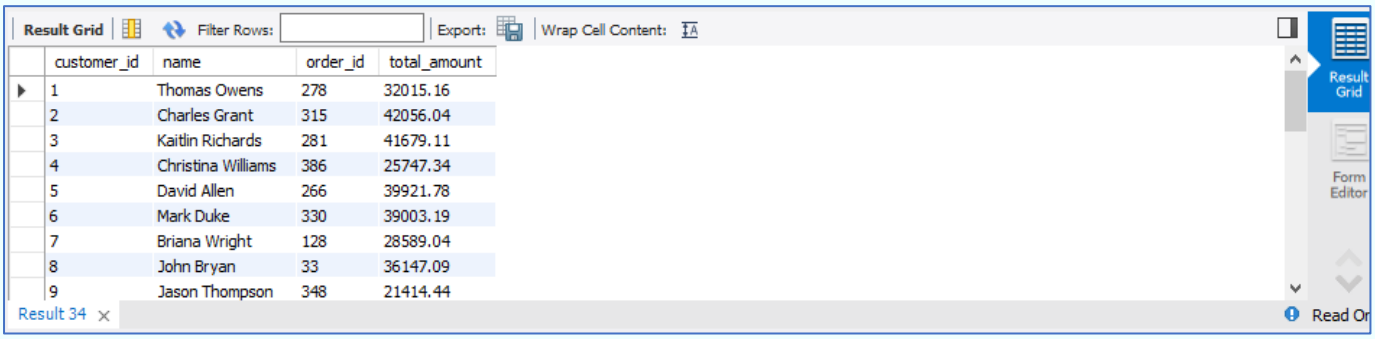
```
SELECT o.customer_id,
      o.order_id,
      o.total_amount
FROM orders o
WHERE o.total_amount = (
   SELECT MAX(o2.total_amount)
   FROM orders o2
   WHERE o2.customer_id = o.customer_id
)
ORDER BY o.customer_id;
```



customer_id	order_id	total_amount
1	278	32015.16
2	315	42056.04
3	281	41679.11
4	386	25747.34
5	266	39921.78
6	330	39003.19

7. Highest Order Per Customer (Including Names)

```
SELECT c.customer_id, c.name, o.order_id, o.total_amount
FROM customers c
JOIN orders o
   ON c.customer_id = o.customer_id
WHERE o.total_amount = (
        SELECT MAX(o2.total_amount)
        FROM orders o2
        WHERE o2.customer_id = o.customer_id
)
ORDER BY c.customer_id;
```

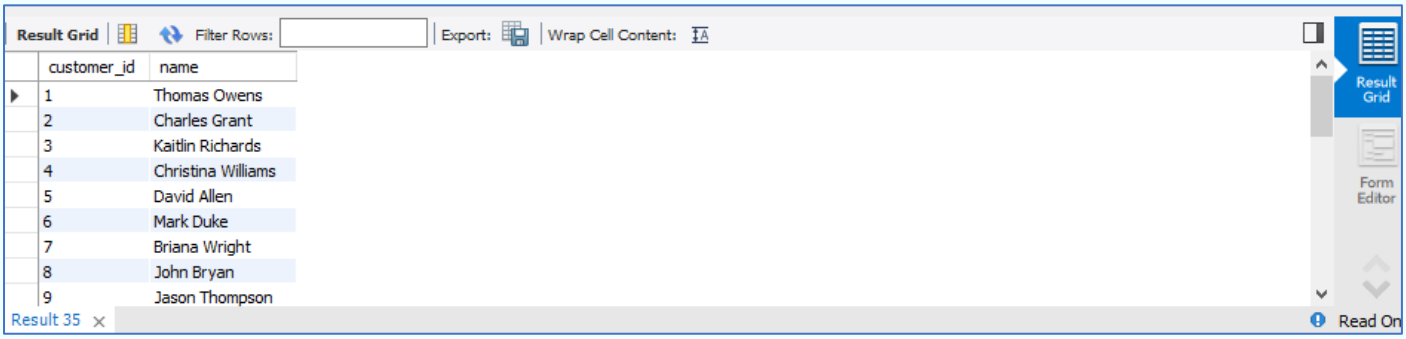


customer_id	name	order_id	total_amount
1	Thomas Owens	278	32015.16
2	Charles Grant	315	42056.04
3	Kaitlin Richards	281	41679.11
4	Christina Williams	386	25747.34
5	David Allen	266	39921.78
6	Mark Duke	330	39003.19
7	Briana Wright	128	28589.04
8	John Bryan	33	36147.09
9	Jason Thompson	348	21414.44

LEVEL 6: SET OPERATIONS

1. List all customers who have either placed an order or written a product review

```
SELECT c.customer_id, c.name
FROM customers c
JOIN orders o
ON c.customer_id = o.customer_id
UNION
SELECT c.customer_id, c.name
FROM customers c
JOIN product_reviews pr
ON c.customer_id = pr.customer_id;
```



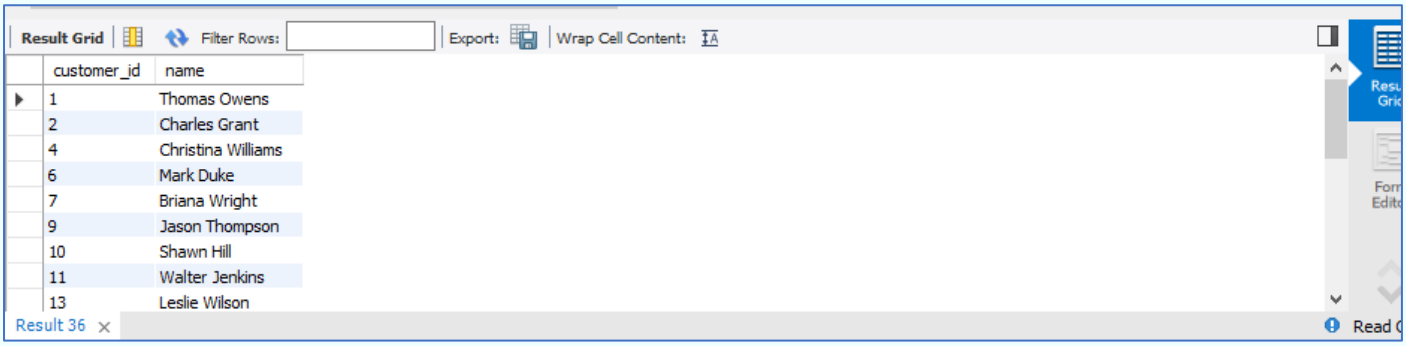
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	name
1	Thomas Owens
2	Charles Grant
3	Kaitlin Richards
4	Christina Williams
5	David Allen
6	Mark Duke
7	Briana Wright
8	John Bryan
9	Jason Thompson

Result 35

2. List all customers who have placed an order as well as reviewed a product [intersect notsupported]

```
SELECT DISTINCT c.customer_id, c.name
FROM customers c
JOIN orders o
ON c.customer_id = o.customer_id
JOIN product_reviews pr
ON c.customer_id = pr.customer_id;
```



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	name
1	Thomas Owens
2	Charles Grant
4	Christina Williams
6	Mark Duke
7	Briana Wright
9	Jason Thompson
10	Shawn Hill
11	Walter Jenkins
13	Leslie Wilson

Result 36